

Installing OpenAI Codex CLI on Pawsey Setonix (pawsey1308)

Project installation guide

Target system: Pawsey Setonix

Project: pawsey1308

Architecture: Linux x86_64

Validated: 8 October 2026

Validated versions: Node.js v24.21.0, npm 11.19.0, Codex CLI 0.161.0

This is a project-level installation guide for users of pawsey1308. It is based on a working Setonix installation and is designed around Pawsey's filesystem layout and multi-login-node environment. It is not an official Pawsey or OpenAI policy document.

1. Design of the installation

Codex CLI is installed **per user**. The installation is split deliberately across Setonix filesystems.

COMPONENT	LOCATION	WHY
Node.js runtime	<code>/software/projects/pawsey1308/\$USER/apps/</code>	Persistent user-installed software
Codex CLI	<code>/software/projects/pawsey1308/\$USER/apps/npm-global/</code>	Persistent user-installed software
Codex config/auth/session files	<code>/software/projects/pawsey1308/\$USER/.codex/</code>	Persistent and private
Codex SQLite databases	<code>/scratch/pawsey1308/\$USER/.cache/codex/sqlite/\$HOSTNAME/</code>	Keeps active SQLite/WAL files away from <code>/software</code> ; separates login nodes
npm cache	<code>/scratch/pawsey1308/\$USER/.cache/npm/</code>	Disposable and re-creatable
Runtime temporary files	<code>/scratch/pawsey1308/\$USER/tmp/codex/</code>	Disposable and re-creatable
Launcher	<code>\$HOME/bin/codex</code>	Tiny file that makes <code>codex</code> available in SSH sessions

The important principle is:

```

/software -> persistent software and persistent Codex
configuration/authentication
/scratch  -> caches, temporary files, and active SQLite databases
/home     -> only a tiny launcher and normal shell configuration

```

Why not put everything in `$HOME`?

On Setonix, `/home` has a comparatively small quota and file-count limit. Node, npm packages, caches, plugins, and Codex state can create unnecessary pressure on that quota.

Why not put everything in `/scratch`?

Pawsey scratch is temporary and files that are not accessed for long enough may be purged. Authentication, configuration, and the installed software therefore belong in persistent storage.

Why are SQLite files treated specially?

Codex uses several SQLite databases (`state`, `logs`, `memories`, `queue`, `goals`, etc.) and may use WAL files. In testing on Setonix, keeping these directly under the shared `/software` filesystem caused database corruption/rebuild messages.

The tested configuration therefore uses:

```
CODEX_HOME=/software/projects/pawsey1308/$USER/.codex
CODEX_SQLITE_HOME=/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME
```

This keeps persistent Codex files under `/software` while separating active SQLite state by login node.

Trade-off: `/scratch` can be purged. If a per-host SQLite directory is removed after inactivity, Codex can recreate its databases, but local database-backed history/state may be reset. Persistent configuration, authentication, and saved files under `CODEX_HOME` remain in `/software`.

2. Prerequisites

Log in:

```
ssh <pawsey_username>@setonix.pawsey.org.au
```

Check architecture and download tools:

```
uname -m  
which wget  
which curl
```

Expected architecture:

```
x86_64
```

Check whether Pawsey currently provides Node.js:

```
module spider nodejs  
module avail nodejs
```

At the time this guide was validated, no `nodejs` module was available.

Check quota:

```
quota
```

Optional usage checks:

```
du -sh "/software/projects/pawsey1308/$USER" 2>/dev/null  
du -sh "/scratch/pawsey1308/$USER" 2>/dev/null
```

A warning from `quota` about an unrelated protected mount can be ignored if the `/home` and `/software` quota tables still print successfully.

Why this matters

On HPC systems, **file count** can be as important as raw storage capacity. npm caches in particular can create many files, so they should not be stored unnecessarily in `/home` or `/software`.

3. Define project paths

Run:

```
PROJECT=pawsey1308
SOFTWARE="/software/projects/${PROJECT}/${USER}"
SCRATCH="/scratch/${PROJECT}/${USER}"
NODE_VERSION=24.21.0
```

Check:

```
echo "$SOFTWARE"
echo "$SCRATCH"
```

For example:

```
/software/projects/pawsey1308/rhuang
/scratch/pawsey1308/rhuang
```

Why use explicit `pawsey1308` paths?

Some users belong to more than one Pawsey project. `$MYSOFTWARE` and `$MYSCRATCH` can therefore be ambiguous in documentation. Explicit project paths ensure that everyone following this guide installs Codex under `pawsey1308`.

4. Create the required directories

Run:

```
mkdir -p "$SOFTWARE/apps"  
mkdir -p "$SOFTWARE/apps/npm-global"  
mkdir -p "$SOFTWARE/.codex"  
  
mkdir -p "$SCRATCH/.cache/npm"  
mkdir -p "$SCRATCH/.cache/codex/sqlite"  
mkdir -p "$SCRATCH/tmp/codex"  
  
mkdir -p "$HOME/bin"
```

Protect the persistent Codex directory:

```
chmod 700 "$SOFTWARE/.codex"
```

5. Install Node.js into /software

Change to scratch for the download:

```
cd "$SCRATCH"
```

Download Node.js and its checksum list:

```
wget "https://nodejs.org/dist/v${NODE_VERSION}/node-  
v${NODE_VERSION}-linux-x64.tar.xz"  
wget "https://nodejs.org/dist/v${NODE_VERSION}/SHASUMS256.txt"
```

Verify:

```
grep " node-v${NODE_VERSION}-linux-x64.tar.xz" SHASUMS256.txt |  
sha256sum -c -
```

Expected:

```
node-v24.21.0-linux-x64.tar.xz: OK
```

Extract:

```
tar -xf "node-v${NODE_VERSION}-linux-x64.tar.xz"
```

Move Node into persistent software storage:

```
mv "node-v${NODE_VERSION}-linux-x64" \  
"$SOFTWARE/apps/node-v${NODE_VERSION}"
```

Create a stable symlink:

```
ln -sfn "$SOFTWARE/apps/node-v${NODE_VERSION}" \  
"$SOFTWARE/apps/node"
```

Delete the temporary downloads:

```
rm -f "node-v${NODE_VERSION}-linux-x64.tar.xz" SHASUMS256.txt
```

Add Node to `PATH` for the current shell:

```
export PATH="$SOFTWARE/apps/node/bin:$PATH"
```

Verify:

```
which node  
node --version  
npm --version
```

Validated output:

```
/software/projects/pawsey1308/<username>/apps/node/bin/node  
v24.21.0  
11.19.0
```

Why is the `PATH` step necessary?

The npm launcher invokes Node through:

```
/usr/bin/env node
```

Therefore Node must be discoverable on `PATH`; executing Node only by an absolute filename is not enough.

6. Configure npm and Codex paths for the installation shell

Run:

```
export PATH="$SOFTWARE/apps/node/bin:$PATH"

export npm_config_prefix="$SOFTWARE/apps/npm-global"
export npm_config_cache="$SCRATCH/.cache/npm"

export CODEX_HOME="$SOFTWARE/.codex"
export CODEX_SQLITE_HOME="$SCRATCH/.cache/codex/sqlite/$HOSTNAME"

export TMPDIR="$SCRATCH/tmp/codex"
```

Create the current host's SQLite directory:

```
mkdir -p "$CODEX_SQLITE_HOME"
```

Set private permissions:

```
chmod 700 "$CODEX_HOME"
chmod 700 "$CODEX_SQLITE_HOME"
chmod 700 "$npm_config_cache"
chmod 700 "$TMPDIR"
```

Verify:

```
echo "Node: $(which node)"
echo "CODEX_HOME: $CODEX_HOME"
echo "CODEX_SQLITE_HOME: $CODEX_SQLITE_HOME"
echo "npm prefix: $(npm config get prefix)"
echo "npm cache: $(npm config get cache)"
echo "TMPDIR: $TMPDIR"
```

All paths should point into `pawsey1308`.

Why separate `CODEX_HOME` and `CODEX_SQLITE_HOME`?

`CODEX_HOME` contains persistent Codex data and belongs on `/software`.

`CODEX_SQLITE_HOME` controls SQLite-backed state. Separating it keeps the active databases and their WAL/SHM files out of `/software`.

The `$HOSTNAME` suffix prevents different Setonix login nodes from opening the same SQLite database:

```
setonix-01 -> .../sqlite/setonix-01/  
setonix-05 -> .../sqlite/setonix-05/  
setonix-06 -> .../sqlite/setonix-06/
```

This is particularly useful on Setonix because a new SSH connection may land on a different login node.

7. Install Codex CLI

Run:

```
npm install -g @openai/codex@latest
```

Verify using the absolute path:

```
"$SOFTWARE/apps/npm-global/bin/codex" --version
```

Validated example:

```
codex-cli 0.161.0
```

Optional size/file checks:

```
du -sh "$SOFTWARE/apps/npm-global"  
du -sh "$SCRATCH/.cache/npm"  
find "$SOFTWARE/apps/npm-global" -type f | wc -l
```

Do not upgrade npm simply because npm prints an upgrade notice. The npm version bundled with the tested Node release is sufficient unless there is a specific reason to change it.

8. Create the Setonix launcher

The launcher is the most important Setonix-specific part of this installation.

Create:

```
cat > "$HOME/bin/codex" <<'EOF'
#!/bin/bash

umask 077

PROJECT=pawsey1308
SOFTWARE="/software/projects/${PROJECT}/${USER}"
SCRATCH="/scratch/${PROJECT}/${USER}"

export PATH="${SOFTWARE}/apps/node/bin:${PATH}"

export CODEX_HOME="${SOFTWARE}/.codex"
export
CODEX_SQLITE_HOME="${SCRATCH}/.cache/codex/sqlite/${HOSTNAME}"

export npm_config_prefix="${SOFTWARE}/apps/npm-global"
export npm_config_cache="${SCRATCH}/.cache/npm"

export TMPDIR="${SCRATCH}/tmp/codex"

mkdir -p \
    "${CODEX_SQLITE_HOME}" \
    "${npm_config_cache}" \
    "${TMPDIR}"

chmod 700 \
    "${CODEX_HOME}" \
    "${CODEX_SQLITE_HOME}" \
    "${npm_config_cache}" \
    "${TMPDIR}"

exec "${SOFTWARE}/apps/npm-global/bin/codex" \
    --no-daemon \
    "$@"
EOF
```

Make it executable:

```
chmod 700 "$HOME/bin/codex"
```

Why `umask 077`?

Codex databases, logs, configuration, and authentication-related files should not be group- or world-readable. `umask 077` makes newly created files private by default.

Why `--no-daemon`?

Recent Codex releases can use a shared background server. On Setonix this is a poor fit because:

- users may connect to different login nodes;
- Codex persistent files are on shared storage;
- the background daemon itself is tied to a running host/process;
- Codex `0.161.0` was observed to fail on Setonix with:

```
Cannot use the shared background server:  
This session requires api_key_model_discovery to be enabled.
```

Codex itself recommends rerunning with `--no-daemon` in that situation.

For Setonix, this guide therefore deliberately uses a **foreground/no-daemon** Codex session.

9. Make sure `$HOME/bin` is on `PATH`

Check:

```
echo "$PATH"
command -v codex
```

On Setonix, `$HOME/bin` is often already present.

If `command -v codex` does not return:

```
/home/<username>/bin/codex
```

add it to `.bashrc`:

```
grep -qxF 'export PATH="$HOME/bin:$PATH"' "$HOME/.bashrc" \
|| echo 'export PATH="$HOME/bin:$PATH"' >> "$HOME/.bashrc"
```

If required for login shells:

```
grep -qxF 'export PATH="$HOME/bin:$PATH"' "$HOME/.bash_profile"
2>/dev/null \
|| echo 'export PATH="$HOME/bin:$PATH"' >> "$HOME/.bash_profile"
```

Reload only if you changed the shell files:

```
source "$HOME/.bashrc"
```

Test:

```
command -v codex
codex --version
```

Expected:

```
/home/<username>/bin/codex  
codex-cli <version>
```

10. Test a completely fresh SSH session

Exit:

```
exit
```

Reconnect:

```
ssh <pawsey_username>@setonix.pawsey.org.au
```

Without exporting anything manually:

```
command -v codex  
codex --version
```

Then test from the **local computer**:

```
ssh <pawsey_username>@setonix.pawsey.org.au \  
'command -v codex && codex --version'
```

Why this test matters

A setup that works only in the shell where installation variables were exported is incomplete. These tests prove the launcher works for fresh and non-interactive SSH sessions.

11. Sign in

Start Codex:

```
codex
```

For a headless SSH machine, the most convenient authentication method is usually **Device Code**:

1. On the initial sign-in screen, press `Esc`.
2. Choose **Sign in with Device Code**.
3. Codex shows a URL and temporary code.
4. Open the URL on your local computer.
5. Sign in to the desired ChatGPT account.
6. Confirm the device code.
7. Return to Setonix.

Do not share device codes, tokens, or authentication-file contents.

If supported by the installed version, check:

```
codex login status
```

The persistent authentication/configuration is stored under:

```
/software/projects/pawsey1308/$USER/.codex
```

not under purgeable scratch.

12. Verify the SQLite split

After starting and exiting Codex once, run:

```
echo "$HOSTNAME"  
ls -lah "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME"
```

You should see databases similar to:

```
state_5.sqlite  
state_5.sqlite-wal  
state_5.sqlite-shm  
logs_2.sqlite  
logs_2.sqlite-wal  
memories_1.sqlite  
queue_1.sqlite  
goals_1.sqlite
```

This confirms that active SQLite state is being written to scratch.

If Codex was previously run before `CODEX_SQLITE_HOME` was configured, old `*.sqlite*` files may still exist directly under:

```
/software/projects/pawsey1308/$USER/.codex
```

Do **not** immediately delete them. Leave them until the new configuration has been tested successfully on several launches.

13. Test another Setonix login node

Because Setonix can place new SSH sessions on different login nodes, test once from another connection.

Exit and reconnect:

```
exit  
ssh <pawsey_username>@setonix.pawsey.org.au
```

Check:

```
hostname  
codex
```

After exiting Codex:

```
ls -lah "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME"
```

If the new host is, for example, `setonix-01`, the final layout should look like:

```
.../.cache/codex/sqlite/  
├─ setonix-01/  
└─ setonix-05/
```

Each host has its own SQLite database set.

14. Optional: choose a default model and reasoning effort

This is a **user preference**, not a required part of the project installation.

Inside Codex, `/model` can be used to select a model and reasoning level.

The normal persistent config lives at:

```
/software/projects/pawsey1308/$USER/.codex/config.toml
```

For example:

```
model = "gpt-6.1-sol"  
model_reasoning_effort = "medium"
```

However, with Codex CLI `0.161.0`, a fresh TUI session was observed on Setonix to ignore these values in some cases.

If a user wants to **force** a specific startup model, add command-line overrides to the final `exec` command in `$HOME/bin/codex`.

For example, to force **GPT-6.1 Sol / medium**:

```
exec "${SOFTWARE}/apps/npm-global/bin/codex" \  
  --no-daemon \  
  --model gpt-6.1-sol \  
  --config 'model_reasoning_effort="medium"' \  
  "$@"
```

This has higher precedence than `config.toml`.

*Do not add this project-wide unless the project explicitly wants one model for everyone.
It is better treated as a per-user preference.*

15. Optional: redirect Codex temporary helper files

Some Codex versions create temporary helper directories under:

```
$CODEX_HOME/tmp
```

On network filesystems, users may occasionally see:

```
WARNING: failed to clean up stale arg0 temp dirs:
Directory not empty (os error 39)
```

If Codex otherwise starts and works, this warning is not fatal.

If the warning becomes frequent or creates many stale temporary directories, and **no Codex process is running**, a user may redirect the temporary subtree to scratch:

```
mkdir -p "/scratch/pawsey1308/$USER/.cache/codex/tmp"

rm -rf "/software/projects/pawsey1308/$USER/.codex/tmp"

ln -s "/scratch/pawsey1308/$USER/.cache/codex/tmp" \
    "/software/projects/pawsey1308/$USER/.codex/tmp"
```

Verify:

```
ls -ld "/software/projects/pawsey1308/$USER/.codex/tmp"
```

This workaround is optional; `CODEX_SQLITE_HOME` is the more important Setonix-specific change.

16. ChatGPT Desktop SSH connection

If using ChatGPT Desktop's SSH connection feature, first make sure the command works from the local terminal:

```
ssh <pawsey_username>@setonix.pawsey.org.au \  
'command -v codex && codex --version'
```

If that succeeds, the remote CLI installation is visible to an SSH client.

The launcher in `$HOME/bin/codex` is deliberately used so external SSH integrations do not depend on manually loading a module or exporting environment variables first.

17. Normal usage on Setonix

Go to the project/repository you want Codex to work on:

```
cd /path/to/project  
codex
```

Codex is suitable for tasks such as:

- reading and editing source code;
- examining logs and configuration files;
- reviewing Git changes;
- generating analysis scripts;
- writing Slurm job scripts;
- submitting `sbatch` jobs;
- inspecting job output and errors.

Important HPC rule

Do **not** use Codex to run heavy CPU, memory, I/O, multiprocessing, or MUSE/data-cube workloads directly on a login node.

Use Slurm for substantial computation.

For example, ask Codex to prepare an `sbatch` script and submit it rather than launching the heavy program directly.

18. Updating Codex

Use the same npm prefix/cache configuration as the original installation:

```
PROJECT=pawsey1308
SOFTWARE="/software/projects/${PROJECT}/${USER}"
SCRATCH="/scratch/${PROJECT}/${USER}"

export PATH="$SOFTWARE/apps/node/bin:$PATH"
export npm_config_prefix="$SOFTWARE/apps/npm-global"
export npm_config_cache="$SCRATCH/.cache/npm"

npm install -g @openai/codex@latest
codex --version
```

Why?

If `npm_config_prefix` is omitted, npm may install a second copy of Codex somewhere else.

19. Updating Node.js

Install a new Node release alongside the old release rather than overwriting it.

For example:

```
apps/  
├─ node-v24.21.0/  
├─ node-v<NEW_VERSION>/  
└─ node -> node-v<NEW_VERSION>/
```

After installing the new version:

```
ln -sfn "$SOFTWARE/apps/node-v<NEW_VERSION>" \  
    "$SOFTWARE/apps/node"
```

Test:

```
"$SOFTWARE/apps/node/bin/node" --version  
"$SOFTWARE/apps/node/bin/npm" --version  
codex --version
```

Keep the previous Node version until the new one has been tested.

20. Scratch purge behaviour

The following data is deliberately stored in `/scratch`:

```
npm cache  
Codex temporary files  
per-host Codex SQLite databases
```

If scratch contents are purged:

- npm will rebuild its cache;
- temporary directories will be recreated;
- Codex will create new SQLite databases.

However, database-backed local history, memory, queue, or similar local state may be reset if the SQLite directory is removed.

The following remain persistent in `/software`:

```
Node.js  
Codex CLI  
Codex authentication  
config.toml  
persistent Codex files under CODEX_HOME
```

21. Security and permissions

The launcher uses:

```
umask 077
```

and creates the important runtime directories with mode 700.

You can verify:

```
ls -ld \  
  "/software/projects/pawsey1308/$USER/.codex" \  
  "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME" \  
  "/scratch/pawsey1308/$USER/.cache/npm" \  
  "/scratch/pawsey1308/$USER/tmp/codex"
```

For existing SQLite files created before the private `umask` was added:

```
find "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME" \  
  -maxdepth 1 -type f -exec chmod 600 {} \;
```

Do not share:

- `auth.json`;
- access tokens;
- device codes;
- private Codex logs containing sensitive content;
- another user's `CODEX_HOME`.

Every project member should authenticate separately.

22. Troubleshooting

A. `npm` says `/usr/bin/env: 'node': No such file or directory`

Node exists, but its `bin` directory is not on `PATH`.

Run:

```
export
PATH="/software/projects/pawsey1308/$USER/apps/node/bin:$PATH"
```

Then:

```
node --version
npm --version
```

B. `codex: command not found`

Check:

```
ls -l "$HOME/bin/codex"
echo "$PATH"
command -v codex
```

`$HOME/bin` must be on `PATH`.

C. Shared background server / `api_key_model_discovery` error

Example:

```
Cannot use the shared background server:
This session requires api_key_model_discovery to be enabled.
```

The Setonix launcher should already contain:

```
--no-daemon
```

Check:

```
tail -n 10 "$HOME/bin/codex"
```

If `--no-daemon` is missing, add it to the final `exec` command.

D. Codex says the local database is damaged

Example:

```
Codex couldn't start because its local database appears to be
damaged.
...
file is not a database
```

First verify the launcher contains:

```
export
CODEX_SQLITE_HOME="${SCRATCH}/.cache/codex/sqlite/${HOSTNAME}"
```

Then check:

```
ls -lah "/scratch/pawsey1308/${USER}/.cache/codex/sqlite/${HOSTNAME}"
```

If Codex was previously using SQLite files directly in `/software`, it may rebuild once during the migration. After `CODEX_SQLITE_HOME` is active, new databases should appear under scratch.

Do not copy a known-corrupted SQLite database into the new location.

E. failed to clean up stale arg0 temp dirs

Example:

```
WARNING: failed to clean up stale arg0 temp dirs:  
Directory not empty (os error 39)
```

If Codex still starts, this is not fatal.

If needed, use the optional `$CODEX_HOME/tmp` -> scratch symlink described in Step 15.

F. The configured model is ignored on startup

If `config.toml` contains:

```
model = "gpt-6.1-sol"  
model_reasoning_effort = "medium"
```

but the TUI starts with another model, add explicit startup options to the launcher:

```
--model gpt-6.1-sol \  
--config 'model_reasoning_effort="medium" '
```

Command-line options take precedence over the config file.

G. Scratch cache or SQLite state disappeared

This can happen because scratch is temporary.

The launcher recreates the directories automatically. Codex may rebuild its SQLite databases.

Persistent authentication and configuration remain under:

```
/software/projects/pawsey1308/$USER/.codex
```

H. Quota problems

Check:

```
quota

du -sh "/software/projects/pawsey1308/$USER"
du -sh "/scratch/pawsey1308/$USER"

find "/software/projects/pawsey1308/$USER" -xdev -type f | wc -l
```

Remember that file-count quota may be the limiting factor.

23. Final verification checklist

Run on Setonix:

```
command -v codex
codex --version

/software/projects/pawsey1308/$USER/apps/node/bin/node --version
/software/projects/pawsey1308/$USER/apps/node/bin/npm --version

ls -ld "/software/projects/pawsey1308/$USER/.codex"
ls -ld "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME"
```

After one Codex session:

```
ls -lah "/scratch/pawsey1308/$USER/.cache/codex/sqlite/$HOSTNAME"
```

From the local computer:

```
ssh <pawsey_username>@setonix.pawsey.org.au \
    'command -v codex && codex --version'
```

A validated layout should resemble:

```
/software/projects/pawsey1308/<username>/
├─ apps/
│   ├── node-v24.21.0/
│   ├── node -> node-v24.21.0
│   └─ npm-global/
└─ .codex/
    ├── config.toml
    ├── authentication/persistent state
    └─ other persistent Codex files

/scratch/pawsey1308/<username>/
├─ .cache/
│   ├── npm/
│   └─ codex/
│       └─ sqlite/
```

```
|      |— setonix-01/
|      |— setonix-05/
|      |— ...
└─ tmp/
    └─ codex/
```

```
/home/<username>/
└─ bin/
    └─ codex
```

24. Known-good launcher

For convenience, this is the complete generic launcher validated for `pawsey1308`:

```
#!/bin/bash

umask 077

PROJECT=pawsey1308
SOFTWARE="/software/projects/${PROJECT}/${USER}"
SCRATCH="/scratch/${PROJECT}/${USER}"

export PATH="${SOFTWARE}/apps/node/bin:${PATH}"

export CODEX_HOME="${SOFTWARE}/.codex"
export
CODEX_SQLITE_HOME="${SCRATCH}/.cache/codex/sqlite/${HOSTNAME}"

export npm_config_prefix="${SOFTWARE}/apps/npm-global"
export npm_config_cache="${SCRATCH}/.cache/npm"

export TMPDIR="${SCRATCH}/tmp/codex"

mkdir -p \
    "${CODEX_SQLITE_HOME}" \
    "${npm_config_cache}" \
    "${TMPDIR}"

chmod 700 \
    "${CODEX_HOME}" \
    "${CODEX_SQLITE_HOME}" \
    "${npm_config_cache}" \
    "${TMPDIR}"

exec "${SOFTWARE}/apps/npm-global/bin/codex" \
    --no-daemon \
    "$@"
```

Optional per-user model pin:

```
exec "${SOFTWARE}/apps/npm-global/bin/codex" \  
  --no-daemon \  
  --model gpt-6.1-sol \  
  --config 'model_reasoning_effort="medium"' \  
"$@"
```

25. Reference sources

Pawsey:

- Setonix General Information:
<https://pawsey.atlassian.net/wiki/spaces/US/pages/51929028/Setonix%2BGeneral%2BInformation>
- Filesystems and their Use:
<https://pawsey.atlassian.net/wiki/spaces/US/pages/51925876>
- Setonix Software Environment:
<https://pawsey.atlassian.net/wiki/spaces/US/pages/51929054/Setonix%2BSoftware%2BEnvironment>

OpenAI / Codex:

- Using Codex with a ChatGPT plan:
<https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan>
- OpenAI Codex repository:
<https://github.com/openai/codex>
- `--no-daemon` background-server implementation/work:
<https://github.com/openai/codex/pull/46088>
- Example Codex issue showing `--no-daemon` as the supported fallback when the shared background server cannot run:
<https://github.com/openai/codex/issues/48043>
- `CODEX_SQLITE_HOME` behaviour / SQLite-home example:
<https://github.com/openai/codex/issues/29953>
- Discussion of configurable SQLite state on shared systems:
<https://github.com/openai/codex/issues/23168>

Node.js:

- Node.js v24.21.0 archive:
<https://nodejs.org/en/download/archive/v24.21.0>
-

26. Maintenance note

This guide pins Node.js `24.21.0` because that was the version validated on Setonix on 8 October 2026.

Codex is installed with:

```
npm install -g @openai/codex@latest
```

so future users may receive a newer Codex version with different behaviour.

When updating this project guide, re-test at least:

1. `codex --version`;
2. fresh interactive SSH startup;
3. non-interactive SSH discovery;
4. ChatGPT/device-code login;
5. `--no-daemon`;
6. `CODEX_SQLITE_HOME`;
7. startup from more than one Setonix login node;
8. model-selection behaviour if the project documents a preferred model;
9. Slurm-safe usage for any substantial computation.

Revision notes: v2

Compared with the first draft, this version incorporates behaviour actually observed during installation and testing on Setonix:

- added `--no-daemon` as the project default;
- added per-login-node `CODEX_SQLITE_HOME` under scratch;
- documented the SQLite corruption/rebuild issue seen on `/software`;
- added `umask 077` and stricter directory permissions;
- made model pinning optional and documented the CLI override workaround;
- moved the `$CODEX_HOME/tmp` symlink workaround to optional troubleshooting rather than the core install;

- added cross-login-node validation;
- clarified the persistence trade-off of storing SQLite state on Pawsey scratch.